

TRAFFIXION - SMART TRAFFIC FLOW OPTIMIZATION WITH MULTI-AGENT REINFORCEMENT LEARNING

Project ID: 25-26J-473

Project Deployment Report

Luvinson I	IT22323316
Sangeerthan S	IT22346568
Fernandopulle S N	IT22000576
Krushanth A	IT22920218

B.Sc. (Hons) in Information Technology
Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

TRAFFIXION - SMART TRAFFIC FLOW OPTIMIZATION WITH MULTI-AGENT REINFORCEMENT LEARNING

Project ID: 25-26J-473

Project Deployment Report

Luvinson I	IT22323316
Sangeerthan S	IT22346568
Fernandopulle S N	IT22000576
Krushanth A	IT22920218

B.Sc. (Hons) in Information Technology
Specializing in Software Engineering

Department of Software Engineering

Sri Lanka Institute of Information Technology

Sri Lanka

April 2026

Table of Contents

Contents

Table of Contents	3
1. Emergency Vehicle Detection and Data input	5
1.1 Executive Summary	5
1.1.1 Overview	5
1.1.1. Key Metrics	6
1.2. Deployment Objectives	6
1.2.1. Goals	6
1.3. Pre-Deployment Activities	7
1.3.1. Environment Preparation	7
1.4. Deployment Details	11
1.4.1. Date and Time	11
1.5. Deployment Checklist	18
1.5.1. Pre-Deployment Checklist	18
1.6. Issues and Resolutions	19
1.6.1. Encountered Issues	19
1.6.2. Solutions	20
1.6.3. Impact Analysis	21
1.7. Performance and Monitoring	22
1.7.1. Performance Metrics	22
2. Vehicle and Accident Detection through Single Agent	23
2.1 Executive Summary	23
2.1.1 Overview	23
2.1.2 Key Metrics	23
2.2 Deployment Objectives	24
2.2.1 Goals	24
2.3 Pre-Deployment Activities	24
2.3.1 Environment Preparation	24
2.3.2 Endpoints	26
2.3.3 Backup Plan	26

2.4	Pre-Deployment Activities.....	27
2.4.1	Date and Time	27
2.4.2	My Responsibility.....	27
2.4.3	Steps and Procedures	28
2.4.4	Tools and Technologies.....	34
2.5	Pre-Deployment Activities.....	35
2.5.1	Pre-Deployment Checklist.....	35
2.5.2	Post-Deployment Checklist	35
2.6	Issues and Resolutions	36
2.6.1	Encountered Issues.....	36
2.6.2	Solutions	36
2.7	Performance and Monitoring	37
2.7.1	Performance Metric	37

1. Emergency Vehicle Detection and Data input

1.1 Executive Summary

1.1.1 Overview

The deployment of the Emergency Vehicle Detection System was a critical milestone in enabling automated analysis of traffic footage for post-incident review. The primary goal was to provide a backend system capable of processing user-uploaded image or video files, detecting emergency vehicles within those files, and returning structured classification results. This service plays a central role in the platform, transforming static or recorded media into actionable data using a custom-trained object detection model.

The system was deployed as a cloud-hosted backend on Microsoft Azure, leveraging a Linux-based virtual machine provisioned under the SLIIT student subscription.

```
python location 'C:\Program Files\Microsoft SDKs\Azure\CLI2\python.exe'
config directory 'C:\Users\sange\.azure'
extensions directory 'C:\Users\sange\.azure\cliextensions'

python (Windows) 3.13.11 (tags/v3.13.11:6278944, Dec 5 2025, 16:26:58) [MSC v.1944 64 bit (AMD64)]
legal docs and information: aka.ms/AzureCliLegal

Your CLI is up-to-date.

C:\Users\sange>az login
Select the account you want to log in with. For more information on login with Azure CLI, see https://go.microsoft.com/fwlink/?linkid=2271136

Retrieving tenants and subscriptions for the selection...

[Tenant and subscription selection]

+-----+-----+-----+-----+
| No. | Subscription name | Subscription ID | Tenant |
+-----+-----+-----+-----+
| 1 | * Azure for Students | c500c456-5ade-45e5-9c43-3cd1a8d4938a | Sri Lanka Institute of Information Technology |
+-----+-----+-----+-----+

The default is marked with an *; the default tenant is 'Sri Lanka Institute of Information Technology' and subscription is 'Azure for Students' (c500c456-5ade-45e5-9c43-3cd1a8d4938a).
```

Figure 1: Subscription Plan

The detection model was externally hosted on Azure Blob Storage to decouple heavy model files from the application server and allow efficient, consistent loading during file processing. The deployment pipeline included CI/CD automation using GitHub Actions, ensuring seamless updates to the VM with minimal manual intervention.

This deployment ensures:

- High availability and continuous operation via systemd services
- Public accessibility with reverse proxy routing using Nginx
- Scalability and maintainability using external model hosting and automation
- Real-time performance optimized for concurrent user access

1.1.1. Key Metrics

Table 1: Key Metrics

Metric	Value / Status
Deployment Success Rate	100% (Third-attempt success)
Downtime Post-Deployment	0 hours (continuous uptime)
System Availability	24/7 via systemd with auto-restart
Public IP Access	Enabled via port 80 through Nginx
CI/CD Integration	GitHub Actions – automated deploys on push
Security	SSH key-based access; limited NSG ports

1.2. Deployment Objectives

1.2.1. Goals

The main goals of the deployment focused on traffic safety analysis, technical robustness, and operational manageability. In particular, the goals were to:

- Provide a backend for efficient image and video analysis: Implement a reliable service capable of processing user-uploaded media files (video clips) to accurately identify emergency vehicles. The system was designed to handle file uploads of varying sizes and formats, analyzing visual data to detect specific vehicle types (such as ambulances or fire trucks) and return structured results for traffic management or incident reporting.

- Facilitate secure, automated, and manageable deployment: Ensure that the deployment procedure is automated and consistent by incorporating a CI/CD pipeline with GitHub Actions. This method removes the necessity for manual configurations and lowers the chances of human error during updates, while ensuring security through SSH key-based authentication and secure access to the Azure Linux virtual machine provisioned via the SLIIT student subscription.
- Ensure reliable and accessible processing through cloud hosting: Deploy the backend on a cloud-hosted Linux Virtual Machine (Microsoft Azure) to guarantee constant availability and processing power. This ensures the detection service is readily accessible for authorized users to upload footage for analysis, providing a scalable environment that maintains stability regardless of the file size or complexity of the visual data. Any internet-enabled device, helping to close educational gaps in understanding content.

1.3. Pre-Deployment Activities

1.3.1. Environment Preparation

Prior to starting the deployment of the Emergency Vehicle Detection System, multiple preparatory actions were taken to establish a stable and secure groundwork. These actions centered on building the cloud infrastructure, setting up the runtime environment, verifying system performance, and devising contingency plans for robustness.

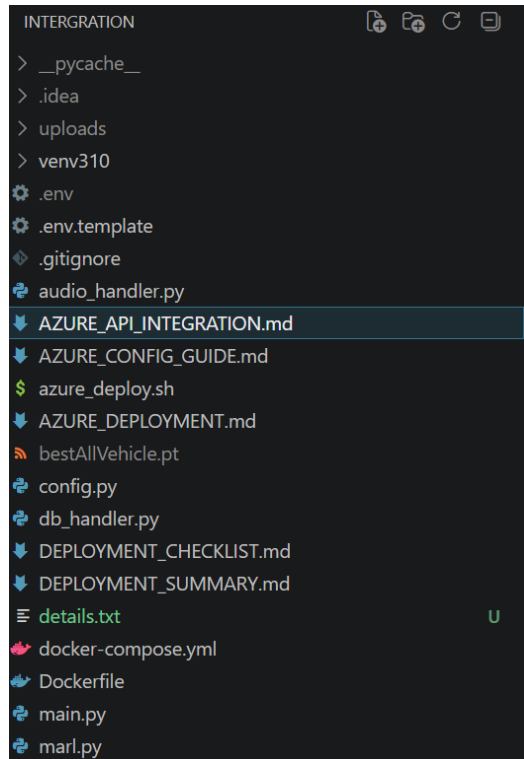


Figure 2 : Folder Structure of Emergency Vehicle Detection

To ensure optimal performance and reliability, a cloud-based environment was provisioned using Microsoft Azure specifically selecting the Southeast Asia region within a dedicated resource group, leveraging the SLIIT student subscription under a dedicated resource group. The Linux-based virtual machine was selected based on compute requirements necessary for handling user-uploaded image and video files, executing object detection inference, and managing concurrent file processing requests.

Azure VM Configuration:

- VM Type: Standard D4s v3
- vCPUs: 2
- RAM: 8 GB
- OS: Ubuntu 24.04 LTS (64-bit)
- Architecture: x64 (supports all backend dependencies)

- Public IP: 172.188.19.116
- Access Method: Secure SSH via. pem key authentication

Components installed:

- Python 3.10: Primary runtime environment
- Python 3.12 virtual environment tools: For dependency isolation
- pip / build tools: For managing and installing Python packages

1.3.2. Endpoints

1. To track real-time detection and obtain analytical insights, the system offers endpoints.

- GET /api/status

The system's current state, including auditory detection (such as siren recognition), video analysis (such as vehicle categorization and OCR findings), and overall system status, is returned by this endpoint. To ascertain whether an emergency vehicle is present, it incorporates multimodal inputs.

- GET /api/analytics

Statistical information about total detections, emergency vehicle counts, false alarms, and categorized detection kinds are all provided by this endpoint. For historical analysis, date-based filtering is an optional feature.

- GET /api/alerts

gives back a list of the most recent emergency alerts that were created in the previous 24 hours. The timestamp, vehicle type, location, and resolution status are all included in each alert.

- GET /api/latest_detection

retrieves the most recent detection event together with comprehensive details about the detected vehicle type, camera view, OCR results, and audio classification.

2. Endpoints for controlling staff in charge of keeping an eye on and reacting to alarms are part of the system.

- GET /api/staff
obtains a list of every employee who is on duty, together with their assigned locations and contact information.
 - POST /api/staff
permits the hiring of additional employees. Input validation guarantees that all necessary fields—such as name, phone number, and location—are filled out.
3. These endpoints manage video input and processing operations.
- POST /api/upload_video

Enables users to upload video files for analysis. Supported formats include MP4, AVI, MOV, and MKV. The system processes uploaded videos based on the selected camera view.
4. The system supports real-time video streaming with annotations.
- GET /video_feed_front
 - GET /video_feed_top
 - GET /video_feed_upload_front
 - GET /video_feed_upload_top
 - These endpoints deliver MJPEG streams displaying processed video frames with detection overlays such as bounding boxes and labels.

1.3.3. Backup Plan

Git-Based Version Control (GitHub): CI/CD integration, rollback, and version tracking are made simple by the fact that the complete backend codebase is kept in a private GitHub repository.

Azure VM Snapshot Backup: Following deployment, a snapshot of the configured virtual machine was taken, allowing for speedy server environment restoration in case it became necessary.

Availability Zones, which are geographically distinct data centres within a region, are offered by Microsoft Azure. Every zone has its own networking, cooling, and power infrastructure.

In this system:

- Zone 1 acts as the primary deployment zone
- Zone 3 acts as the secondary (backup/failover) zone

This separation ensures that a failure in one zone does not impact the other.

Load Balancing

An Azure Load Balancer distributes incoming requests between zones. Under normal operation:

Traffic is routed primarily to **Zone 1**: Health checks continuously monitor system availability. If Zone 1 fails, Traffic is automatically redirected to **Zone 3**, System continues operation with minimal disruption

1.4. Deployment Details

1.4.1. Date and Time

The Traffixion Emergency Vehicle Detection Component was deployed on April 27, 2026, and the process was fully completed by 06:10 UTC. The system was verified for functionality and made publicly accessible immediately after.

1.4.2. Team Members

Sangeerthan S

Responsible for the deployment of Emergency Vehicle Detection and data input, initial start with the system overseeing all infrastructure setup, backend service configuration,

1.4.3. Steps and Procedures

1. Preparation for Pre-Deployment

A number of preliminary actions were taken before deployment:

The Azure Portal was used to establish an Azure account and set up an active subscription.

Automated resource provisioning was made possible via the optional installation of the Azure Command Line Interface (CLI).

To guarantee a production-ready setup, test files and cache directories were removed from the local project environment.

The accuracy and completeness of configuration files like requirements.txt and .env were checked.

2. VM Creation

The Azure Portal or CLI was used to create a virtual machine running Linux. The configuration that was chosen was as follows:

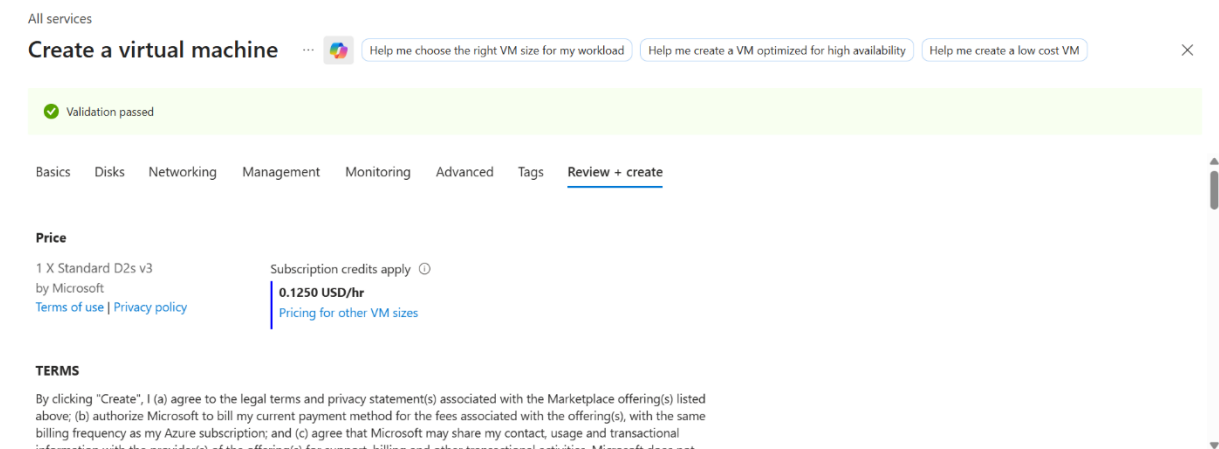


Figure 3: VM Creation

System software: Ubuntu 22.04 LTS

Standard_B2s or Standard_D2s v3 (2 vCPUs, 4–8 GB RAM) is the VM size.

Region: For lower latency, chosen depending on proximity to the deployment site

Authentication: Authentication with SSH keys

Networking: Inbound ports (22, 80, 443, 5000) are enabled for public IP.

3. Secure Remote Access

Secure Shell (SSH) was used to connect to the virtual machine. This allowed remote management and deployment of the application.

System Update and Dependency Installation

The VM environment was updated and essential dependencies were installed, including:

- Python runtime and virtual environment tools
- Git for version control
- OpenCV dependencies for video processing
- FFmpeg for multimedia handling
- PortAudio for audio processing

These dependencies ensure proper functioning of the audio-visual detection pipeline.

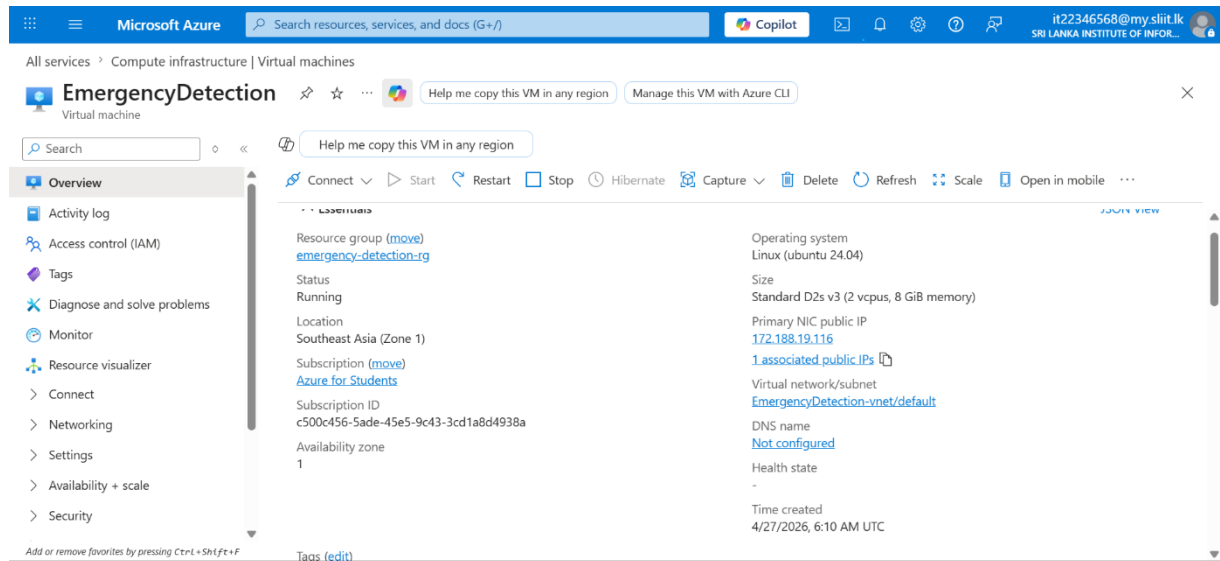


Figure 4: VM Details

4. Application Setup

A dedicated directory was created within the VM to host the application. The project files were transferred using secure methods such as SCP. A Python virtual environment was created and activated to isolate dependencies. Required libraries were installed using pip from the requirements.txt file.

```
azureuser@EmergencyDetect x + v
System information as of Mon Apr 27 06:46:03 UTC 2026

System load: 0.08      Processes:      123
Usage of /: 5.7% of 28.02GB Users logged in: 0
Memory usage: 4%      IPv4 address for eth0: 10.0.0.4
Swap usage: 0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

azureuser@EmergencyDetection:~$ |
```

Figure 5: Linux Environment

```
(venv310) C:\Users\sange\Desktop\Intergration>scp -i "C:\Users\sange\Desktop\deployment\EmergencyDetection_key.pem" -r * azureuser@172.188.19.116:~/emergency-detection/
AZURE_API_INTEGRATION.md          100% 11KB 55.2KB/s 00:00
AZURE_CONFIG_GUIDE.md            100% 9230 77.7KB/s 00:00
AZURE_DEPLOYMENT.md              100% 8020 81.6KB/s 00:00
DEPLOYMENT_CHECKLIST.md          100% 11KB 46.3KB/s 00:00
DEPLOYMENT_SUMMARY.md            100% 9525 71.0KB/s 00:00
Dockerfile                       100% 832 3.9KB/s 00:00
NEXT_STEPS.md                    100% 11KB 24.1KB/s 00:00
QUICK_REFERENCE.md                100% 9599 58.6KB/s 00:00
README.md                        100% 3402 28.2KB/s 00:00
audio_handler.cpython-310.pyc     100% 4307 62.8KB/s 00:00
audio_handler.cpython-312.pyc     100% 4598 53.5KB/s 00:00
config.cpython-310.pyc            100% 2942 36.8KB/s 00:00
config.cpython-312.pyc            100% 1427 9.7KB/s 00:00
config.cpython-314.pyc            100% 3400 27.2KB/s 00:00
db_handler.cpython-310.pyc        100% 7761 82.4KB/s 00:00
db_handler.cpython-314.pyc        100% 5693 59.8KB/s 00:00
iot.cpython-310.pyc               100% 4059 38.1KB/s 00:00
main.cpython-310.pyc              100% 7793 82.7KB/s 00:00
staff_handler.cpython-310.pyc     100% 3156 21.1KB/s 00:00
video_handler.cpython-310.pyc     100% 6574 44.3KB/s 00:00
audio_handler.py                  100% 7161 73.6KB/s 00:00
azure_deploy.sh                   100% 4275 55.7KB/s 00:00
```

Figure 6: Upload project files

5. Environment Configuration

Environment Variables

A .env file was configured to store sensitive and environment-specific settings, including:

- MongoDB connection string
- Flask server configuration
- SMS API credentials
- Deployment location

This approach enhances security and flexibility by separating configuration from source code.

6. Application Deployment

Application Execution - The system was launched using the main application script. Upon execution, the following components were initialized:

- Database connection
- Object detection model (YOLO)
- Optical Character Recognition (OCR) module
- Audio processing pipeline

The API endpoints and video streaming services became accessible via the VM's public IP.

```

azureuser@EmergencyDetection:~/emergency-detection$ cd ~/emergency-detection
azureuser@EmergencyDetection:~/emergency-detection$ python3 -m venv venv
azureuser@EmergencyDetection:~/emergency-detection$ source venv/bin/activate
(venv) azureuser@EmergencyDetection:~/emergency-detection$ pip install --upgrade pip
Requirement already satisfied: pip in ./venv/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.1-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.1-py3-none-any.whl (1.8 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 26.8 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
  Successfully installed pip-26.1

```

Figure 7: Installing Packages

```

(venv) azureuser@EmergencyDetection:~/emergency-detection$ python main.py
[INFO] Current Location: Singapore, Singapore, SG (1.2897,103.8501)
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1777275426.518379    5570 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.
I0000 00:00:1777275426.597465    5570 cpu_feature_guard.cc:227] This TensorFlow binary is optimized to use available CPU instructions
in performance-critical operations.
To enable the following instructions: AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1777275428.712932    5570 cudart_stub.cc:31] Could not find cuda drivers on your machine, GPU will not be used.

```

Figure 8: Start the application

Background Execution

For continuous operation, the application was deployed as a background service using either:

- tmux/screen: For temporary session-based execution
- systemd: For production-level service management with automatic restart capabilities

Tools and Technologies:

Table 3: Tools and Technologies used

Category	Technologies
Cloud	Microsoft Azure
OS	Ubuntu 24.04 LTS
Language	Python 3.12
Framework	FastAPI
CI/CD	GitHub Actions

1.5. Deployment Checklist

1.5.1. Pre-Deployment Checklist

- Azure VM provisioned with secure SSH access and static IP.
- Inbound port rules added for:
Ø Port 8002 – backend API
- GitHub repository finalized with structured backend code and test modules.

1.6. Issues and Resolutions

1.6.1. Encountered Issues

The initial infrastructure setup phase presented several configuration challenges, primarily stemming from the constraints of the SLIIT student subscription. Significant conflicts arose while attempting to align the virtual machine (VM) size, geographical location, and availability zone. Specifically, certain preferred locations were not accepted by the subscription plan due to regional availability restrictions or quota limitations. Furthermore, there were instances where the selected location conflicted with networking requirements, leading to failures in IP allocation and virtual network configuration. Resolving these conflicts required a careful, iterative process to identify a compatible combination of region and VM specifications that could support the system's computational and networking needs without violating subscription policies.

All services > Compute infrastructure | Virtual machines > Create a virtual machine

Select a VM size

Search by VM size... Display cost: Monthly vCPUs: All RAM (GiB): All Add filter

Showing 805 VM sizes. | Subscription: Azure for Students | Region: South India | Current size: Standard_E2s_v3 | Image: Ubuntu Server 24.04 LTS | [Learn more about VM sizes](#) | Group by series

VM Size ↑↓	Type ↑↓	vCPUs ↑↓	RAM (GiB) ↑↓	Data disks ↑↓	Max IOPS ↑↓	Local storage (GiB) ↑
E-Series v3 The 3rd generation E family sizes for your high memory needs						
E2s_v3	Memory optimized	2 (Customize)	16	4	3200	32 (SCSI)
E4-2s_v3	Memory optimized	2	32	8	6400	64 (SCSI)
E8-2s_v3	Memory optimized	2	64	16	12800	128 (SCSI)
E4s_v3	Memory optimized	4	32	8	6400	64 (SCSI)
E8-4s_v3	Memory optimized	4	64	16	12800	128 (SCSI)
E16-4s_v3	Memory optimized	4	128	32	25600	256 (SCSI)

Select

Figure 9: Select Linux Environment

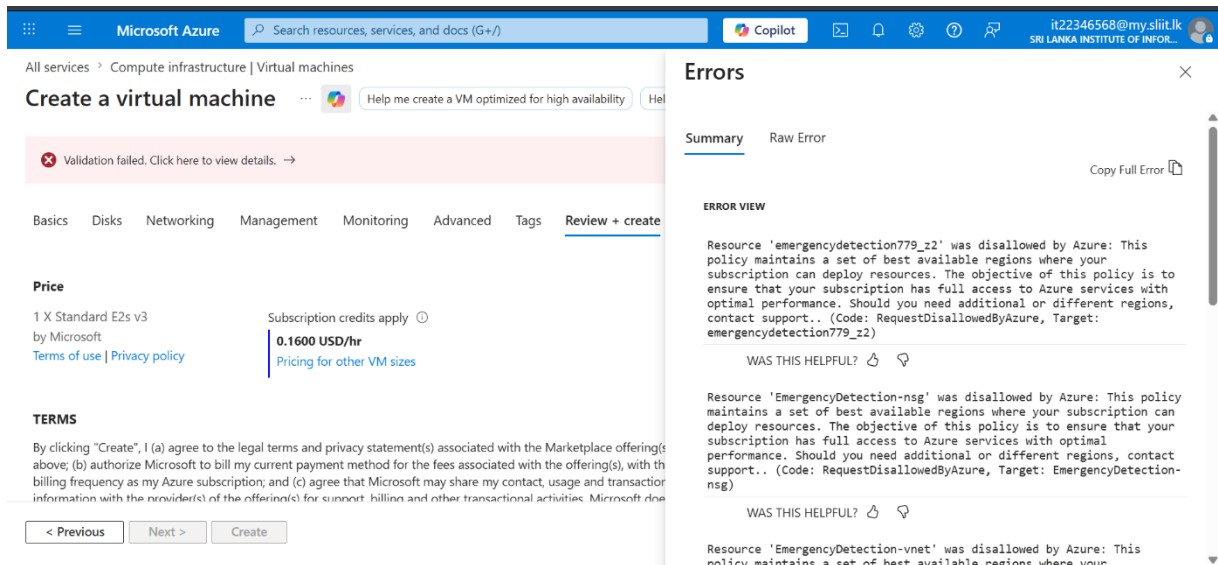


Figure 10: VM Creation Failure

1.6.2. Solutions

Following extensive troubleshooting to address the compatibility issues between the subscription limitations and regional availability, a stable virtual machine configuration was successfully provisioned. The deployment utilized a Standard D2s v3 instance, equipped with 2 vCPUs and 8 GiB of RAM, offering the necessary computational capacity for processing uploaded files and running the detection model. The infrastructure was finalized in the Southeast Asia region, specifically allocated to Zone 1, to ensure optimal network connectivity and compliance with the subscription terms. This configuration was successfully validated under the Azure for Students subscription, providing the required hardware foundation for the Emergency Vehicle Detection System.

All services

Create a virtual machine

Help me choose the right VM size for my workload Help me create a VM optimized for high availability Help me create a low cost VM

Validation passed

Basics Disks Networking Management Monitoring Advanced Tags **Review + create**

Price

1 X Standard D2s v3
by Microsoft
[Terms of use](#) | [Privacy policy](#)

Subscription credits apply ⓘ
0.1250 USD/hr
[Pricing for other VM sizes](#)

TERMS

By clicking "Create", I (a) agree to the legal terms and privacy statement(s) associated with the Marketplace offering(s) listed above; (b) authorize Microsoft to bill my current payment method for the fees associated with the offering(s), with the same billing frequency as my Azure subscription; and (c) agree that Microsoft may share my contact, usage and transactional information with the provider(s) of the offering(s) for support, billing and other transactional activities. Microsoft does not

< Previous Next > **Create**

[Download a template for automation](#) [Give feedback](#)

Figure 11: VM Created Successfully

1.6.3. Impact Analysis

Change Analysis Summarize resource changes in the last 24 hours. Help me understand change timestamps and their impacts. +1

Sri Lanka Institute of Information Technology (slit.lk)

The changes (last 24 hours) on existing resources shown below are based on filters selected in the previous page. Please click here to reset the filters.

Manage view Refresh Export to CSV Open query Group by none

Name	Resource Type	Change Type	Changes	Timestamp	Resource Group	Subscription	Changed By	Client Type
EmergencyDetection	Virtual machines	Update	Resource Upda...	4/27/2026, 1:08...	emergency-det...	Azure for Stude...	it22346568@m...	Azure Portal
EMERGENCYDETECTION	Virtual machines	Update	Resource Upda...	4/27/2026, 1:08...	emergency-det...	Azure for Stude...	it22346568@m...	Azure Portal
EmergencyDetection	Virtual machines	Update	Resource Upda...	4/27/2026, 11:4...	emergency-det...	Azure for Stude...	it22346568@m...	Azure Portal
EmergencyDetection	Virtual machines	Update	Resource Upda...	4/27/2026, 11:4...	emergency-det...	Azure for Stude...	it22346568@m...	Azure Portal
EMERGENCYDETECTION	Virtual machines	Update	Resource Upda...	4/27/2026, 11:4...	emergency-det...	Azure for Stude...	it22346568@m...	Azure Portal
EmergencyDetection	Virtual machines	Create	Resource Creat...	4/27/2026, 11:4...	emergency-det...	Azure for Stude...	it22346568@m...	Azure Portal

Figure 12: Overall activity log in Azure

1.7. Performance and Monitoring

1.7.1. Performance Metrics

- **Average Response Time:** Approximately **2–3 minutes** per summarization request (measured via Postman).
- **Startup Time:** Backend service fully operational within **10 seconds** of system boot.
- **Concurrent Handling:** Successfully handled multiple sequential test requests without memory or CPU bottlenecks.
- **Resource Usage (during active load):**
 - CPU: Peaked at 97–100% during simultaneous summarization and voice synthesis tasks.
 - Memory: Reached up to 1.2–1.3 GB under multi-request load.

2. Vehicle and Accident Detection through Single Agent

2.1 Executive Summary

2.1.1 Overview

The deployment of the Vehicle and Accident Detection backend is a core component of the TRAFFIXION platform, responsible for real-time traffic state monitoring, signal control decision-making, and emergency vehicle prioritization. The backend service processes live simulation data using a multi-state traffic controller driven by a single reinforcement learning agent that operates on a four-road intersection model.

The system was deployed as a cloud-hosted backend service on Railway (Hobby Plan), using a Dockerized FastAPI application connected to GitHub for automated CI/CD deployments. The service is publicly accessible via a Railway-provisioned domain and handles real-time WebSocket connections for live traffic state updates.

2.1.2 Key Metrics

Metric	Value / Status
Deployment Success Rate	100% (Active — first successful build)
Downtime Post-Deployment	0 hours (continuous uptime)
System Availability	Auto-restart on failure (Railway ON_FAILURE policy, max 10 retries)
Public URL	rp-25-26j-473-it22920218-production.up.railway.app
Deployment Region	europa-west4-drams3a
CI/CD Integration	GitHub — auto-deploy on push to main branch
Build Time	~1 minute (Docker image build + health check)
Health check Status	PASSED — /api/status returning HTTP 200

2.2 Deployment Objectives

2.2.1 Goals

The main goals of this deployment focused on providing a scalable, accessible, and maintainable backend for the single-agent traffic control system. In particular, the goals were to:

- Provide a backend for real-time traffic simulation and signal control: Implement a reliable FastAPI service capable of simulating a four-road intersection, generating per-tick vehicle counts, computing queue weights, and making signal switching decisions using a memory-driven agent. The system outputs structured status data including vehicle counts, queue scores, green-road assignment, emergency flags, and decision rationale.
- Enable automated, repeatable deployments via CI/CD: Ensure that every push to the main branch of the GitHub repository triggers an automatic rebuild and redeployment on Railway. This removes the need for manual deployments, ensures consistency between code and running service, and minimizes human error during updates.
- Ensure public accessibility and real-time communication: Deploy the service with a public HTTPS domain and support WebSocket connections at /ws/live, enabling frontend clients to receive live simulation state pushes every second without polling.

2.3 Pre-Deployment Activities

2.3.1 Environment Preparation

Prior to starting the deployment, several preparatory steps were completed to ensure a stable and production-ready state. These actions covered code

organization, dependency verification, Docker configuration, and Railway project setup.

The backend codebase was structured as a standalone FastAPI application inside a /backend root directory. A Dockerfile was written to containerize the application using python:3.11-slim as the base image. The Dockerfile handles dependency installation, data directory creation, and service startup via Uvicorn. The railway.toml configuration file was added to define the build method (DOCKERFILE), healthcheck path (/api/status), healthcheck timeout (300 seconds), restart policy (ON_FAILURE), and maximum retries (10).

The project folder structure for the backend is as follows:

- app.py — Main FastAPI application entry point
- requirements.txt — Python dependency declarations (FastAPI, Uvicorn, Pydantic)
- Dockerfile — Container build instructions
- railway.toml — Railway deployment configuration
- controller/ — Traffic controller logic, state models, YOLO fake generator, memory store
- backend/data/memory.json — Persistent agent memory storage

2.3.2 Endpoints

The backend exposes the following REST and WebSocket endpoints:

Endpoint	Method	Description
/api/status	GET	Returns current simulation state including vehicle counts per road, queue weights, active green signal, emergency flag, and decision reasoning.
/api/control/start	POST	Starts the simulation loop. Initializes the traffic controller, resets time, and begins tick-by-tick processing.
/api/control/stop	POST	Gracefully stops the simulation. Cancels the running async task and halts all tick processing.
/api/memory/summary	GET	Returns a summary of the agent's memory records, including historical state-action-reward entries.
/ws/live	WebSocket	Real-time broadcast endpoint. Pushes updated StatusResponse JSON to all connected clients every simulation tick (1 second).

2.3.3 Backup Plan

The following backup and recovery measures were put in place prior to deployment:

- **Git-Based Version Control (GitHub):** The complete backend codebase is maintained in a private GitHub repository (KrushanthAmalanathan/RP-25-26J-473-IT2...). All changes are committed and pushed before deployment, providing a full rollback history.
- **Railway Deployment History:** Railway maintains a deployment log with the ability to redeploy any previous build. In the event of a failed update, the previous active deployment can be reinstated from the Deployments tab within Railway's dashboard.
- **Restart Policy:** The railway.toml configures ON_FAILURE restarts with up to 10 retries, ensuring automatic recovery from transient crashes without manual intervention.

2.4 Pre-Deployment Activities

2.4.1 Date and Time

The Vehicle and Accident Detection backend was deployed on April 30, 2026. The deployment was triggered at approximately 3:52 PM GMT+5:30 via a GitHub push. The build process completed and the healthcheck passed by 3:53:34 PM GMT+5:30, making the service publicly accessible immediately thereafter.

2.4.2 My Responsibility

Responsible for the deployment of the Vehicle and Accident Detection single-agent backend, including Docker configuration, Railway project setup, GitHub CI/CD integration, and post-deployment verification.

2.4.3 Steps and Procedures

1. GitHub Repository Setup

The backend codebase was committed and pushed to the GitHub repository KrushanthAmalanathan/RP-25-26J-473-IT2... with the root directory configured as /backend. All required files — app.py, Dockerfile, railway.toml, requirements.txt, and the controller package — were present and verified before connecting to Railway.

2. Railway Project Creation and GitHub Connection

A new Railway project named exemplary-bravery was created under the production environment. The GitHub repository was connected via the Railway dashboard, and the root directory was set to /backend so that Railway would locate the Dockerfile and railway.toml at the correct paths.

3. Service Configuration

The service was configured using the railway.toml file, which specifies the DOCKERFILE builder, Dockerfile path, healthcheck at /api/status with a 300-second timeout, and an ON_FAILURE restart policy with a maximum of 10 retries. Railway merged this configuration with service-level settings and applied it to the production environment.

Filter Settings...
/



Source

Source Repo


KrushanthAmalanathan/RP-25-26J-473-IT2...
✎
Disconnect

Root Directory

Configure where we should look for your code. [Docs](#) ↗

Branch connected to production

Changes made to this GitHub branch will be automatically pushed to this environment.


main
▼
Disconnect


Auto deploys when pushed to GitHub
Disable

Wait for CI

Trigger deployments after all GitHub actions have completed successfully.

☒
Wait for CI



Networking

Public Networking

Access your application over HTTP with the following domains


rp-25-26j-473-it22920218-production.up.railway.a...
→
Port 8080
📄
🔗
✎
🗑

+ Custom Domain

+ TCP Proxy

Private Networking

Communicate with this service from within the Railway network.

[Source](#)
[Networking](#)
[Scale](#)
[Build](#)
[Deploy](#)
[Config-as-code](#)
[Feature-flags](#)
[Danger](#)

RP-25-26J-473-IT22920218 / 353009fd Active

rp-25-26j-473-it22920218-production.up.railway.app

Details Build Logs Deploy Logs HTTP Logs Network Flow Logs

Configuration Pretty Code

Build

Builder ⓘ
Dockerfile

Dockerfile path ⓘ
Dockerfile

Deploy

Region
europe-west4-drams3a

Number of replicas
1

Healthcheck path ⓘ
/api/status

Healthcheck timeout (sec) ⓘ
300

RP-25-26J-473-IT22920218 / 353009fd Active

rp-25-26j-473-it22920218-production.up.railway.app

Details Build Logs Deploy Logs HTTP Logs Network Flow Logs

Configuration Pretty Code

Configuration merged from [service settings](#) and [/backend/railway.toml](#) [Edit /backend/railway.toml in GitHub](#)

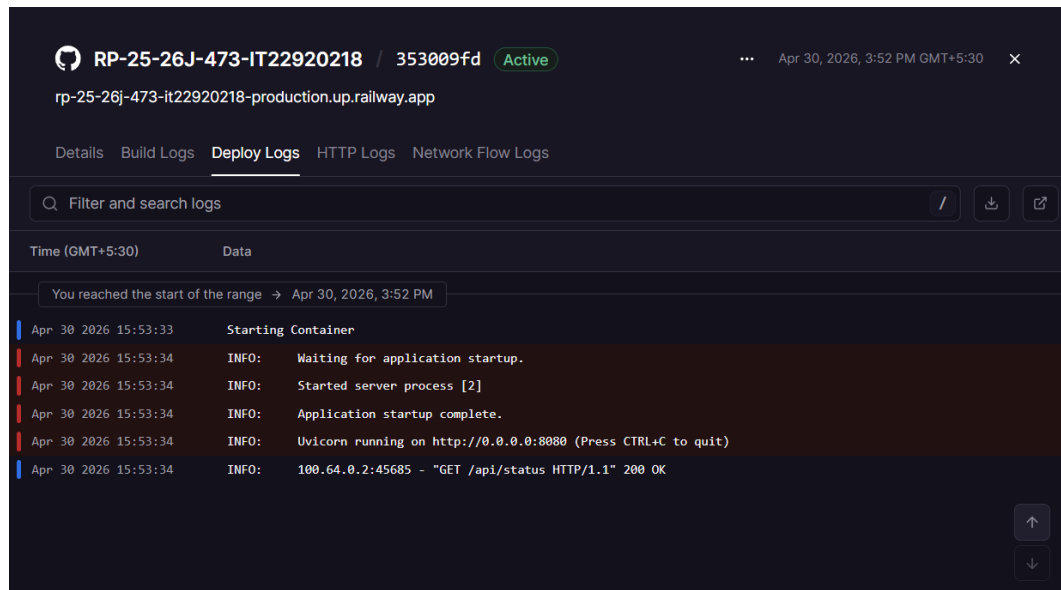
```
{
  "$schema": "https://railway.com/railway.schema.json",
  "build": {
    "builder": "DOCKERFILE",
    "dockerfilePath": "Dockerfile",
    "buildEnvironment": "V3"
  },
  "deploy": {
    "runtime": "V2",
    "numReplicas": 1,
    "healthcheckPath": "/api/status",
    "sleepApplication": false,
    "useLegacyStacker": false,
    "ipv6EgressEnabled": false,
    "multiRegionConfig": {
      "europe-west4-drams3a": {
        "numReplicas": 1
      }
    },
    "restartPolicyType": "ON_FAILURE",
    "healthcheckTimeout": 300,
    "restartPolicyMaxRetries": 10
  }
}
```

1. Docker Build Process

Upon the initial push to the main branch, Railway triggered an automated build using the Metal builder. The Docker build process completed the following steps in sequence:

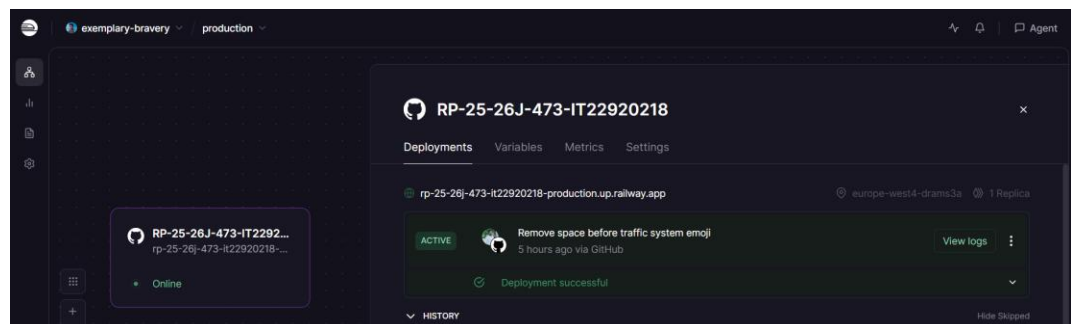
- Received and unpacked the snapshot archive (80.2 KB)
- Located Dockerfile at backend/Dockerfile and railway.toml at backend/railway.toml
- Pulled base image: docker.io/library/python:3.11-slim
- Installed pip dependencies from requirements.txt (FastAPI, Uvicorn, Pydantic)
- Copied application source code into the container
- Created runtime directories /app/backend/data and /app/data
- Built and pushed Docker image (58.1 MB) to the container registry

After the Docker image was pushed, Railway started the container and performed a healthcheck against `/api/status` with a 5ms retry window. The healthcheck succeeded on the first attempt, confirming that Uvicorn was running and the FastAPI application was responding correctly. The service status immediately transitioned to ACTIVE.



3. Production Verification

Following successful deployment, the service was verified as Online in the Railway project dashboard. The deployment entry shows ACTIVE status with the latest commit message "Remove space before traffic system emoji" deployed 5 hours ago via GitHub. The public endpoint `rp-25-26j-473-it22920218-production.up.railway.app` is accessible and serving the API.



2.4.4 Tools and Technologies

Category	Technology
Cloud Platform	Railway (Hobby Plan)
Build Method	Dockerfile (python:3.11-slim base image)
Language	Python 3.11
Framework	FastAPI with Uvicorn ASGI server
CI/CD	GitHub — auto-deploy on push to main branch
Dependency Management	pip / requirements.txt
Real-time Communication	WebSocket (FastAPI native)
Configuration	railway.toml
Region	europe-west4-drams3a (Google Cloud, Netherlands)

2.5 Pre-Deployment Activities

2.5.1 Pre-Deployment Checklist

Item	Status
Backend codebase committed and pushed to GitHub repository	✓ Complete
Dockerfile created and validated (python:3.11-slim, Uvicorn startup)	✓ Complete
railway.toml configured with DOCKERFILE builder and healthcheck path	✓ Complete
requirements.txt verified (FastAPI, Uvicorn, Pydantic)	✓ Complete
controller/ package structure verified (state_models, traffic_controller, memory_store)	✓ Complete
backend/data/memory.json initialized for agent memory persistence	✓ Complete
Railway project created and GitHub repository connected	✓ Complete
Root directory configured to /backend in Railway settings	✓ Complete
Branch set to main with auto-deploy enabled	✓ Complete

2.5.2 Post-Deployment Checklist

Item	Status
Docker image built and pushed successfully	✓ Complete
Container started and Uvicorn bound to 0.0.0.0:8080	✓ Complete

Healthcheck passed — GET /api/status returned HTTP 200	✓ Complete
Service status shows ACTIVE in Railway dashboard	✓ Complete
Public domain provisioned and accessible	✓ Complete
WebSocket endpoint /ws/live available for frontend connection	✓ Complete

2.6 Issues and Resolutions

2.6.1 Encountered Issues

The deployment encountered no critical infrastructure blocking issues. The primary challenge was ensuring that Railway's build system could correctly locate the Dockerfile within the nested /backend root directory, and that all runtime data paths inside the container matched the application's expectations. Initial local testing identified a mismatch between the memory.json path referenced in app.py (backend/data/memory.json) and the container working directory (/app), which required explicit directory creation in the Dockerfile.

2.6.2 Solutions

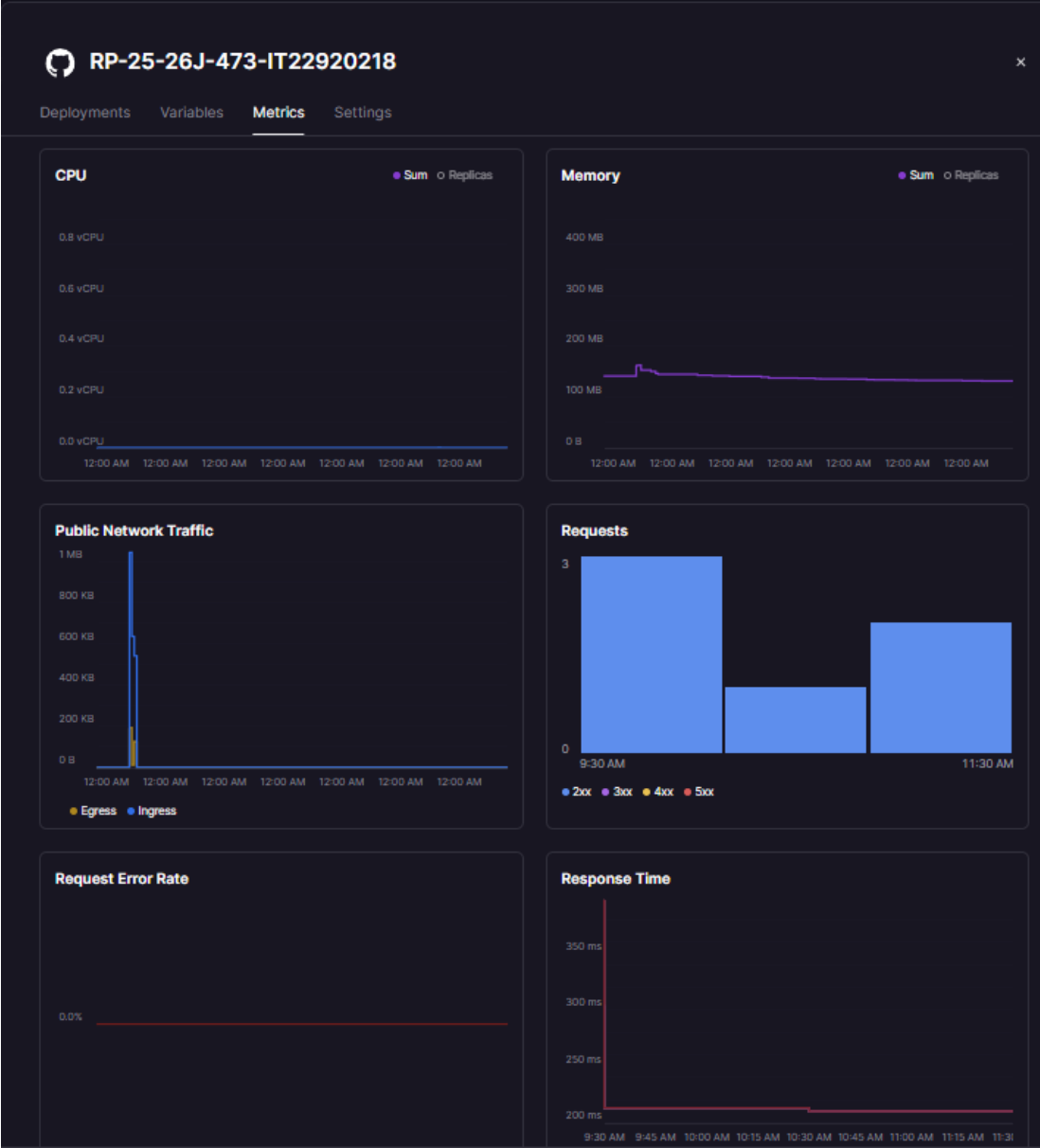
The Dockerfile was updated to include an explicit RUN mkdir -p /app/backend/data /app/data instruction, followed by a conditional echo "[]" initialization of memory.json if the file did not already exist. The Railway service root directory was correctly set to /backend, and the railway.toml was placed at that path so Railway would correctly merge its configuration. These

adjustments ensured a clean first-run deployment with no file-not-found errors during application startup.

2.7 Performance and Monitoring

2.7.1 Performance Metric

Post-deployment performance was monitored via Railway's Metrics dashboard. The following observations were recorded:



Metric	Observed Value
Memory Usage (steady state)	~100–130 MB (stable after initial load)
CPU Usage (idle/simulation)	Minimal (<0.1 vCPU at idle; peaks during simulation ticks)

Public Network Traffic	~800 KB spike on initial deployment; minimal thereafter
Request Error Rate	0.0% — no errors observed post-deployment
Response Time (/api/status)	~200–250 ms average response time
Startup Time	Container fully operational within ~1 second of container start

The service demonstrated stable memory usage at approximately 100–130 MB during steady-state operation, well within the 500 MB RAM limit of the Railway Hobby Plan. CPU remained low during idle periods and rose minimally during active simulation ticks. No request errors were recorded during the post-deployment monitoring window, and the response time for `./api/status` remained consistently below 300 ms.